Contents

1 Documentation technique - SAE Application intelligente **4**
1.1 Table des matières 4

2 Accueil **5**

3 SAE - Application intelligente **5**
3.1 Documentation technique 5
3.2 CI et déploiement 5

4 Présentation **5**

5 Présentation du code source **5**
5.1 Objet 5
5.2 Organisation 5
5.3 Structure principale 6
5.4 Rôle de cette documentation 6
5.5 À retenir 6

6 init **6**

7 Module git_commit_release_notes_generator **6**
7.1 Rôle 6
7.2 Sous-modules 6
7.3 Objectif fonctionnel 6

8 config **7**

9 Configuration **7**
9.1 Description 7
9.2 Variables principales 7
9.3 Emplacement et chargement du fichier `.env` 7
9.4 Points de vigilance 7
9.5 Exemple d'usage 7

10 models **8**

11 Modèles de données **8**
11.1 Description 8
11.2 Structures principales 8
11.2.1 DiffFile 8
11.2.2 CommitMessage 8
11.3 Rôle dans le flux 8

12 init **8**

13 Package ollama_client **8**
13.1 Rôle 9
13.2 Sous-modules 9
13.3 Objectif 9

14 base **9**

15 Base Ollama client **9**
15.1 Description 9

15.2 Rôle attendu . . . 9
15.3 Utilisation . . . 9

16 embedding **9**

17 Embedding Ollama **9**
17.1 Description . . . 10
17.2 Statut actuel . . . 10
17.3 Rôle fonctionnel attendu . . . 10

18 exceptions **10**

19 Exceptions Ollama **10**
19.1 Description . . . 10
19.2 Exceptions prévues . . . 10
19.3 Pourquoi les centraliser ? . . . 10

20 llm **10**

21 Client LLM **10**
21.1 Description . . . 11
21.2 Flux de travail . . . 11
21.3 Points clés . . . 11
21.4 Fonctions exposées . . . 11
21.5 Dépendances . . . 11

22 mock_llm **12**

23 Module de Simulation de l'IA (Mock LLM) **12**
23.1 Description . . . 12
23.2 Responsabilités . . . 12
23.3 Fonctions exposées . . . 12
23.4 Points clés . . . 12

24 init **13**

25 Service du générateur **13**
25.1 Rôle . . . 13
25.2 Sous-modules . . . 13
25.3 Objectif . . . 13

26 core **13**

27 Core du service (Point d'entrée du pont Go ↔ Python) **13**
27.1 Description . . . 13
27.2 Contrat d'interface (Règle d'or Go / Python) . . . 13
27.3 Les 9 actions gérées (`--action <NOM>`) . . . 14
27.4 Fonctions clés . . . 14

28 utils **15**

29 Utilitaires de service **15**
29.1 Description . . . 15
29.2 Rôle attendu . . . 15
29.3 À compléter . . . 15

30 Présentation **15**

31 Présentation des tests **15**
31.1 Objet . . . 15
31.2 Types de tests présents . . . 15
31.3 Organisation . . . 16
31.4 Rôle de cette documentation . . . 16
31.5 À retenir . . . 16

32 Git wrapper **16**

33 Tests du wrapper Git **16**
33.1 Description . . . 16
33.2 Ce qui est testé . . . 16
33.3 Cas de test principaux . . . 16
33.3.1 1. Parsing d'un diff multi-fichiers . . . 16
33.3.2 2. Sanitisation des secrets . . . 17
33.3.3 3. Structure du payload JSON . . . 17
33.4 Points de vigilance . . . 17

34 Ollama LLM **17**

35 Tests du client Ollama **17**
35.1 Description . . . 17
35.2 Ce qui est testé . . . 17
35.3 Stratégie de test . . . 17
35.4 Cas importants . . . 18
35.4.1 génération réussie au premier essai . . . 18
35.4.2 retry puis succès . . . 18
35.4.3 validation stricte du message . . . 18
35.5 Points de vigilance . . . 18

36 Mock IA **18**

37 Tests du Mock IA (Simulation Hors-Ligne) **18**
37.1 Description . . . 18
37.2 Ce qui est testé . . . 18
37.3 Stratégie de test . . . 19
37.4 Exécution des tests . . . 19

38 Golden diffs **19**

39 Fixtures Golden Diff **19**
39.1 Description . . . 19
39.2 Structure de données . . . 19
39.3 Rôle fonctionnel . . . 19
39.4 Exemple de scénarios . . . 20
39.5 Points de vigilance . . . 20

40 Initialisation **20**

41 Activation des situations de test **20**
41.1 Description . . . 20
41.2 Rôle fonctionnel . . . 20
41.3 Comportement . . . 20
41.4 Utilité dans le projet . . . 21

42 Désinitialisation **21**

43 Neutralisation des situations de test **21**
43.1 Description . . . 21
43.2 Rôle fonctionnel . . . 21
43.3 Objectif . . . 21
43.4 Comportement . . . 21
43.5 Utilité du projet . . . 21

44 Déploiement de la documentation **22**

45 Déploiement de la documentation **22**
45.1 Objectif . . . 22
45.2 Fichiers concernés . . . 22
45.3 Configuration MkDocs . . . 22
45.4 Prérequis : activation manuelle de GitHub Pages (une seule fois) . . . 22
45.5 Prérequis du PDF versionné . . . 22
45.6 Workflow de déploiement . . . 23
45.6.1 Déclenchement . . . 23
45.6.2 Marqueur de version de développement . . . 23
45.6.3 Étapes du job `build` . . . 23
45.6.4 Étapes du job `deploy` . . . 23
45.7 Résultat attendu . . . 23
45.8 Bonnes pratiques . . . 23
45.9 Commande de vérification locale . . . 24

46 Tests automatiques via GitHub Actions **24**

47 Tests automatisés via GitHub Actions **24**
47.1 Objectif . . . 24
47.2 Workflows disponibles . . . 24
47.3 Workflow CI . . . 24
47.3.1 Jobs . . . 24
47.4 Vérification locale . . . 25
47.5 Intérêt du workflow CI . . . 25

48 Sprint 00 **25**

49 Sprint N°1 : Mise en place du projet **25**

50 Sprint 01 **25**

51 Sprint N°2 : Intégration et développement des fonctionnalités **25**

1 Documentation technique - SAE Application intelligente

1.1 Table des matières

Contents

---

2 Accueil

3 SAE - Application intelligente

Documentation du projet d'application intelligente.

Cette documentation présente l'architecture technique, les tests, le suivi de projet et les mécanismes de CI/CD associés.

3.1 Documentation technique

La documentation technique suit la structure du code source et des tests.

- `doc_src_files` : documentation des fichiers du dossier `src`
- `doc_test_files` : documentation des fichiers du dossier `test`
- `monitoring` : suivi des sprints et de l'avancement du projet

3.2 CI et déploiement

Le projet met en place deux éléments essentiels de documentation opérationnelle :

- le déploiement de la documentation : explication du build MkDocs et de la publication sur GitHub Pages
- les tests automatiques via GitHub Actions : explication du lancement des tests et de leur organisation dans la CI

---

4 Présentation

5 Présentation du code source

Cette section regroupe la documentation technique du code applicatif du projet.

5.1 Objet

Elle permet de suivre la structure du dossier `src/` et de comprendre le rôle de chaque module, en gardant une correspondance claire entre le code et sa documentation.

5.2 Organisation

Le dossier `src/` contient le cœur applicatif du projet, notamment :

- la génération des messages de commit
- l'intégration avec Ollama
- la gestion Git et des diff
- les composants CLI
- les services de synthèse et d'analyse

5.3 Structure principale

- `git_commit_release_notes_generator/` : logique métier du projet
- `ollama_client/` : intégration avec les modèles LLM
- `service/` : services de traitement, Git et utilitaires
- `cli/` : interface en ligne de commande et vues utilisateur

5.4 Rôle de cette documentation

Chaque fichier de code a un équivalent Markdown dans ce dossier, avec la même organisation arborescente. Cela permet de :

- retrouver rapidement la description d'un module
- comprendre les responsabilités de chaque composant
- maintenir la documentation synchronisée avec l'évolution du code

5.5 À retenir

La documentation du code source doit rester alignée sur l'implémentation réelle. Quand un module est ajouté, renommé ou modifié, la documentation associée doit être mise à jour dans ce dossier.

---

6 init

7 Module git_commit_release_notes_generator

Fichier source associé : src/git_commit_release_notes_generator/**init**.py

7.1 Rôle

Ce package regroupe le cœur de la génération des messages de commit et des notes de release. Il centralise les données de configuration, les structures de données partagées et les clients externes qui interagissent avec Ollama.

7.2 Sous-modules

- config.md : paramètres de configuration de l'application.
- models.md : modèles de données partagés.
- ollama_client : couche de communication avec Ollama.
- service : logique métier et traitements applicatifs.

7.3 Objectif fonctionnel

Le package sert de point d'entrée logique pour :

- analyser un diff Git ;
- filtrer les données sensibles ;
- transmettre un contexte structuré au modèle LLM ;
- générer un message de commit conforme à Conventional Commits ;
- produire éventuellement des éléments exploitables pour des notes de version.

8 config

9 Configuration

Fichier source associé : src/git_commit_release_notes_generator/config.py

9.1 Description

Ce module contient les paramètres globaux utilisés par le client Ollama et le reste de l'application.

9.2 Variables principales

- `OLLAMA_BASE_URL` : URL du serveur Ollama (ex: `http://10.22.28.190:11434` ou local).
- `OLLAMA_MODEL` : modèle utilisé pour la génération des messages de commit (ex: `gemma4:26b`, `gemma4:12b`).
- `OLLAMA_TIMEOUT_S` : délai de requête avant timeout en secondes.
- `APP_LANGUAGE` : langue de l'application et des messages générés (`fr` ou `en`).
- `MOCK_INTERFACE` : activation du mode simulation complète de l'interface en Go (`true` ou `false`).
- `MOCK_AI` : activation de la simulation des réponses de l'IA Ollama hors-ligne (`true` ou `false`).

9.3 Emplacement et chargement du fichier `.env`

- **Emplacement strict** : Le fichier `.env` réside **exclusivement à la racine du dépôt du projet** (`SAE_sujet_8_application_intelligente/.env`), ou à l'emplacement indiqué par la variable d'environnement système `SMART_COMMIT_APP_ROOT`.
- **Aucune pollution** : Le module ne lit ni n'écrit jamais de `.env` dans `src/` ou dans le répertoire de travail courant `Path.cwd()`, évitant toute confusion ou parasitage d'un dépôt Git tiers analysé par l'outil.
- **Surcharge** : Les variables définies directement dans l'environnement système ont priorité sur le contenu du fichier `.env`.

9.4 Points de vigilance

- La configuration est lue dynamiquement sans nécessiter de recompilation ni de modification de code.
- Les valeurs par défaut restent compatibles avec l'environnement de l'IUT et la CI.
- Le fichier modèle `.env.exemple` est versionné à la racine pour guider le déploiement initial.

9.5 Exemple d'usage

Le client LLM et le service `core.py` importent cette configuration pour construire les requêtes HTTP vers Ollama et configurer la langue de traitement.

10 models

11 Modèles de données

Fichier source associé : src/git_commit_release_notes_generator/models.py

11.1 Description

Ce module définit les structures de données transverses utilisées entre les composants du projet.

11.2 Structures principales

11.2.1 DiffFile

Représente un fichier modifié dans un diff Git.

Champs importants :

- path : chemin du fichier ;
- status : état du fichier (A, M, D, R, etc.) ;
- added / removed : nombre de lignes ajoutées et supprimées ;
- is_binary : indique si le fichier est binaire ;
- patch : contenu brut du diff.

11.2.2 CommitMessage

Représente le message final de commit produit par le modèle.

Champs importants :

- type : type Conventional Commits (feat, fix, docs, chore, etc.) ;
- scope : portée optionnelle ;
- subject : sujet principal ;
- body : description détaillée optionnelle.

11.3 Rôle dans le flux

Les objets de ce module servent de contrat entre :

- le parsing Git ;
- la génération de texte via Ollama ;
- l'assemblage final du message de commit.

Ils évitent les chaînes non structurées et rendent le code plus robuste et testable.

---

12 init

13 Package ollama_client

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/**init**.py

13.1 Rôle

Ce package encapsule la communication avec Ollama pour la génération de messages de commit.

13.2 Sous-modules

- base.md : primitives de base et point d'entrée technique.
- llm.md : logique de génération de message et validation du résultat.
- embedding.md : gestion des embeddings, si cette fonctionnalité est ajoutée plus tard.
- exceptions.md : exceptions métier du client.
- prompts : prompt système utilisé pour imposer la structure du message.

13.3 Objectif

Le package isole les dépendances externes LLM du reste du projet afin de garder la logique métier testable et plus simple à maintenir.

---

14 base

15 Base Ollama client

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/base.py

15.1 Description

Ce module sert de base commune au client Ollama. Il définit les contrats et les éléments de fondation pour les intégrations avec le modèle.

15.2 Rôle attendu

- centraliser les interfaces de communication ;
- préparer les appels HTTP ;
- exposer une structure cohérente pour les autres sous-modules.

15.3 Utilisation

Les modules spécialisés reposent sur cette base pour éviter de dupliquer la logique de transport et de gestion des erreurs.

---

16 embedding

17 Embedding Ollama

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/embedding.py

17.1 Description

Ce module est dédié aux embeddings, c'est-à-dire à la transformation de texte en vecteurs numériques pour les traitements de similarité ou d'indexation.

17.2 Statut actuel

Le fichier est prévu pour évoluer avec les besoins du projet et peut servir à des cas d'usage futurs, notamment la recherche ou la comparaison de diff/commit.

17.3 Rôle fonctionnel attendu

- convertir des chaînes textuelles en représentations vectorielles ;
- collaborer avec d'autres modules de recherche ou de classification ;
- rester isolé de la logique métier principale.

---

18 exceptions

19 Exceptions Ollama

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/exceptions.py

19.1 Description

Ce module définit les erreurs spécifiques à la couche LLM et aux échanges avec Ollama.

19.2 Exceptions prévues

- OllamaCallError : échec de l'appel au serveur.
- CommitMessageValidationError : validation du JSON de réponse impossible ou incohérente.

19.3 Pourquoi les centraliser ?

Cela permet de distinguer clairement :

- les erreurs réseau ;
- les erreurs de format de réponse ;
- les erreurs métier liées au contenu généré.

Cette séparation simplifie le diagnostic et le test des cas limites.

---

20 llm

21 Client LLM

Fichier source associé : src/git_commit_release_notes_generator/ollama_client/llm.py

21.1 Description

Ce module gère la génération du message de commit via Ollama. Il construit le résumé du diff, envoie une requête structurée à l'API de chat, valide la réponse JSON et la transforme en objet métier.

21.2 Flux de travail

1. Rejeter immédiatement une liste de fichiers vide (`ValueError`), sans appel réseau.
2. Lire les fichiers différences et les convertir en résumé exploitable.
3. Construire le prompt système et le message utilisateur.
4. Appeler Ollama via une requête JSON structurée, avec une fenêtre de contexte (`num_ctx`) fixée explicitement.
5. Vérifier que la réponse est bien un JSON conforme.
6. En cas d'erreur, relancer une seconde fois avec le message invalidé en contexte.
7. Convertir la réponse validée en objet CommitMessage.

21.3 Points clés

- format : le modèle répond avec un schéma JSON strict ;
- validation : le code refuse les messages trop longs, absents ou non conformes ;
- robustesse : un retry est tenté pour corriger une réponse invalide ;
- fenêtre de contexte : `options.num_ctx` est fixé explicitement dans la requête (`OLLAMA_NUM_CTX`, config.py) car Ollama utilise sinon une fenêtre par défaut bien plus petite que celle annoncée pour le modèle, ce qui tronquait silencieusement les diffs volumineux ;
- disponibilité : `check_ollama_reachable()` permet de vérifier rapidement (quelques secondes) si le serveur répond, avant de lancer une génération qui pourrait sinon attendre le timeout complet ;
- simulation hors-ligne : si `config.MOCK_AI` est activé, `generate_commit_message` et `check_ollama_reachable` délèguent immédiatement au module `mock_llm` sans aucune requête réseau.

21.4 Fonctions exposées

- `generate_commit_message(diff_files, feedback="")` : fonction principale, décrite ci-dessus.
- `check_ollama_reachable(timeout=3.0, fetch_models=False)` (alias `ping_ollama`) : fonction unique et centralisée de diagnostic Ollama. Vérifie la joignabilité (`/api/version`), mesure la latence réseau en millisecondes et extrait optionnellement les modèles installés (`/api/tags` si `fetch_models=True`). Retourne un objet `PingResult(reachable, error, latency_ms, installed_models)` déballable directement en tuple `(reachable, error)` ou sérialisable via `.to_dict()`. Utilisée de manière transverse par `action_generate_commit`, `action_release_notes` et `action_ping_ollama` (écran de configuration Go).

21.5 Dépendances

- config.md
- models.md
- mock_llm.md
- exceptions.md
- commit_message_system.md

22 mock_llm

23 Module de Simulation de l'IA (Mock LLM)

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/mock_llm.py`

23.1 Description

Ce module fournit une implémentation simulée (Mock) des fonctionnalités d'intelligence artificielle d'Ollama. Il permet de travailler sur l'application en mode hors-ligne sans serveur Ollama actif, tout en conservant l'analyse Git réelle, l'indexation et la persistance des fichiers.

23.2 Responsabilités

- Générer un message de commit déterministe et statique respectant le format Conventional Commits (`feat(mock): commit mocker pour test hors ligne`) ;
- Supporter la prise en compte de consignes utilisateur (`feedback`) injectées dans le corps du message pour les tests de révision interactive ;
- Simuler la charge utile JSON d'un appel à l'API de chat Ollama (`mock_call_chat`) ;
- Générer une réponse fixe et prévisible pour les Release Notes (`mock_release_notes`) ;
- Simuler le ping de connectivité et la liste des modèles installés (`mock_ping`) avec une latence factice de 10 ms.

23.3 Fonctions exposées

Fonction	Arguments	Retour	Description
`mock_generate_commit_message`	`diff_files`, `feedback=""`	`CommitMessage`	Renvoie le commit statique mocké. Lève `ValueError` si la liste de diff est vide.
`mock_call_chat`	`messages`	`dict`	Simule la réponse brute de `/api/chat`.
`mock_release_notes`	`commits`, `from_tag`, `to_tag`	`str`	Renvoie une note de version Markdown statique déterministe.
`mock_ping`	—	`dict`	Renvoie un diagnostic de connectivité simulé réussi avec latence de 10 ms.

23.4 Points clés

- **Simplicité et prévisibilité** : Les réponses sont volontairement statiques et prévisibles afin de faciliter les tests unitaires et le développement hors-ligne.
- **Délégation automatique** : `llm.py` et `core.py` basculent automatiquement vers ce module dès que la variable `MOCK_AI` est activée dans la configuration.

---

24 init

25 Service du générateur

Fichier source associé : src/git_commit_release_notes_generator/service/**init**.py

25.1 Rôle

Ce package rassemble la logique métier pure de l'application. Il sert à orchestrer les traitements et à exposer les fonctions qui manipulent les diff et les données de génération.

25.2 Sous-modules

- core.md : noyau métier du système.
- utils.md : utilitaires de traitement et aide fonctionnelle.

25.3 Objectif

Cette couche permet de garder le code de génération séparé des détails de communication réseau ou de parsing Git.

---

26 core

27 Core du service (Point d'entrée du pont Go ↔ Python)

Fichier source associé : src/git_commit_release_notes_generator/service/core.py

27.1 Description

Ce module constitue le point d'entrée exécutable sous forme de sous-processus par le front-end Go (`src/cli/bridge/python_client.go`). Il orchestre l'ensemble des fonctionnalités du back-end Python : - interaction et extraction des statuts et diffs Git via `GitWrapper` ; - filtrage des fichiers et masquage des secrets ; - interrogation du LLM via `generate_commit_message` ; - synthèse des release notes au format Markdown ; - diagnostic et joignabilité du serveur Ollama.

27.2 Contrat d'interface (Règle d'or Go / Python)

1. **Sortie JSON stricte sur `stdout`** : seul un objet JSON unique est imprimé sur la sortie standard.
2. **Logs et diagnostics sur `stderr`** : toutes les traces, messages d'information et erreurs passent exclusivement par le module `logging` sur `sys.stderr`.
3. **Résilience sans crash** : toutes les exceptions sont interceptées pour renvoyer un JSON d'erreur propre `{"success": false, "error": "..."}`.

27.3 Les 9 actions gérées (`--action <NOM>`)

`--action`	Arguments	Format de sortie JSON	Rôle et source des données
`status`	`--repo`	`{"is_git_repo", "repo_path", "branch", "staged_count", "unstaged_count"}`	Détermine l'état du dépôt Git via `GitWrapper`
`diff`	`--repo`	`{"success", "files": [...], "staged_count"}`	Analyse et assainit les fichiers indexés (`GitWrapper._parse_diff`)
`stage-all`	`--repo`	`{"success", "message"}`	Indexe l'ensemble des fichiers modifiés (`git add -A`)
`generate-commit`	`--repo`, `--feedback`	`{"success", "type", "scope", "subject", "body", "raw_formatted"}`	Inférence du commit Conventional Commits avec support du feedback
`apply-commit`	`--repo`, `--message`	`{"success", "sha", "message"}`	Applique le commit sur la branche courante via `GitWrapper.commit()`
`push`	`--repo`	`{"success", "message"}`	Pousse vers le dépôt distant configuré via `git push`
`release-notes`	`--repo`, `--from`, `--to`	`{"success", "from_tag", "to_tag", "markdown", "commit_count"}`	Synthèse Markdown des commits entre deux révisions (LLM + repli déterministe)
`save-config`	`--base-url`, `--model`, `--timeout`, `--language`	`{"success", "message"}`	Notification de sauvegarde de configuration
`ping-ollama`	—	`{"success", "reachable", "latency_ms", "installed_models"}`	Vérification HTTP de joignabilité et des modèles disponibles sur Ollama

27.4 Fonctions clés

- `action_status(repo_path)` : vérifie la validité du dépôt, gère les branches orphelines (detached HEAD) et dénombre les modifications.
- `action_diff(repo_path)` : expose chaque fichier avec son patch, ses compteurs added/removed, le drapeau `binary` et la détection `has_secrets_masked`.
- `action_generate_commit(repo_path, feedback)` : extrait le diff staged textuel, teste la joignabilité d'Ollama via `check_ollama_reachable()` pour échouer rapidement en cas de problème réseau, et le transmet au module LLM avec la consigne optionnelle utilisateur (ou renvoie le commit statique si `MOCK_AI=True`).
- `action_release_notes(repo_path, from_tag, to_tag)` : extrait l'historique, teste la joignabilité d'Ollama via `check_ollama_reachable()` (échoue immédiatement avec `suc-`

cess=False si injoignable, à l'identique de action_generate_commit) et génère une synthèse catégorisée (Features, Fixes, Refactoring, Docs) via le LLM avec repli déterministe si l'appel chat échoue (ou renvoie la synthèse statique si MOCK_AI=True).
- action_ping_ollama() : teste la joignabilité, mesure la latence et liste les modèles installés en déléguant à la fonction unifiée check_ollama_reachable(fetch_models=True).

---

28 utils

29 Utilitaires de service

Fichier source associé : src/git_commit_release_notes_generator/service/utils.py

29.1 Description

Ce module réunit les fonctions utilitaires destinées à simplifier le traitement, la validation et le filtrage des données transitant entre le diff Git et la génération de message.

29.2 Rôle attendu

- mettre en forme les données ;
- normaliser les entrées ;
- factoriser les blocs de logique réutilisables ;
- isoler les opérations auxiliaires du cœur métier.

29.3 À compléter

Le fichier est vide à l'état actuel ; ce document sert de point d'entrée pour documenter les utilitaires ajoutés ultérieurement.

---

30 Présentation

31 Présentation des tests

Cette section décrit la documentation technique associée aux tests du projet.

31.1 Objet

Le dossier test/ contient les tests unitaires, d'intégration et de situation. Cette documentation permet de comprendre le rôle de chaque test et la stratégie de validation du projet.

31.2 Types de tests présents

- tests unitaires : vérification des fonctions et composants isolés
- tests de wrapper Git : validation de l'interaction avec Git
- tests Ollama LLM : vérification des appels et réponses du modèle
- golden diffs : tests sur des sorties attendues

- tests de situation : scénarios réalistes du projet

31.3 Organisation

La structure des fichiers de documentation suit celle du dossier `test/` pour faciliter la lecture et la maintenance.

31.4 Rôle de cette documentation

Chaque test ou groupe de tests possède une page dédiée afin de :

- expliquer le but du test
- détailler le scénario couvert
- documenter les résultats attendus
- faciliter le suivi qualité du projet

31.5 À retenir

Les tests sont une partie essentielle de la fiabilité du projet. Leur documentation doit rester à jour pour refléter les cas de validation réellement mis en place dans le code.

---

32 Git wrapper

33 Tests du wrapper Git

Fichier source associé : test/test_git_wrapper.py

33.1 Description

Ce module valide le comportement du wrapper Git utilisé pour analyser les diffs et préparer les données envoyées au système de génération de commit.

33.2 Ce qui est testé

- le parsing d'un diff Git en fichiers structurés ;
- la détection des modifications par fichier ;
- la suppression des données sensibles dans les patches ;
- la création d'un payload JSON exploitable par le modèle ou par l'orchestrateur applicatif.

33.3 Cas de test principaux

33.3.1 1. Parsing d'un diff multi-fichiers

Le test vérifie que le diff est découpé correctement en plusieurs objets, avec pour chaque fichier :

- le chemin du fichier ;
- le statut de modification ;
- les lignes ajoutées et supprimées ;
- le patch complet associé.

33.3.2 2. Sanitisation des secrets

Le test simule un contenu contenant une clé ou une valeur sensible et vérifie que le mécanisme de redaction remplace bien les informations sensibles par une valeur de type `[REDACTED_SECRET]`.

33.3.3 3. Structure du payload JSON

Le test s'assure que les objets renvoyés par le wrapper sont convertibles en payloads normalisés, sans informations parasites ou inutiles. Il vérifie notamment la présence des champs attendus comme `path`, `status`, `patch` et `is_binary`.

33.4 Points de vigilance

- La logique de parsing doit rester tolérante aux variations de format de diff Git.
- La redaction des secrets doit être systématique avant toute transmission hors du contexte local.
- Les payloads doivent rester structurés et légers pour éviter d'encombrer le modèle LLM.

---

34 Ollama LLM

35 Tests du client Ollama

Fichier source associé : test/test_ollama_llm.py

35.1 Description

Ce module couvre la logique de validation et d'interaction avec le client Ollama. Il vérifie notamment le comportement du code qui transforme un diff en un message de commit conforme à la norme Conventional Commits.

35.2 Ce qui est testé

- la génération d'un message de commit valide dès le premier essai ;
- le mécanisme de retry lorsqu'une réponse est invalide ;
- l'échec après plusieurs réponses invalides ;
- les règles de validation du message (`type`, `scope`, `subject`) ;
- la vérification de joignabilité rapide et extraction des modèles via `check_ollama_reachable()` / `ping_ollama()` (succès, échec réseau, mesure de latence, récupération des modèles avec `fetch_models`, sérialisation `to_dict`, compatibilité tuple) ;
- la compatibilité avec les anciennes structures de données binaires.

35.3 Stratégie de test

Les appels réseau réels sont évités par remplacement de `urllib.request.urlopen` avec un faux objet de réponse. Cela permet de simuler le comportement de Ollama sans dépendre d'un service distant.

35.4 Cas importants

35.4.1 génération réussie au premier essai

Le test vérifie qu'un payload bien formé est accepté et transformé en objet métier avec les bons champs : type, scope et subject.

35.4.2 retry puis succès

Lorsque la première réponse est invalide, le système réessaie avec une seconde réponse. Cela confirme que la logique de récupération est bien mise en place.

35.4.3 validation stricte du message

Le test contrôle que les sujets terminés par un point, ainsi que les types inconnus, sont rejetés pour rester conforme à la convention de messages attendue.

35.5 Points de vigilance

- La validation doit rester stricte pour éviter des messages de commit peu lisibles ou incohérents.
- Le mécanisme de retry ne doit pas masquer des erreurs de fond ou des réponses non exploitables.
- La compatibilité avec les objets historiques de type `binary` doit être conservée pour ne pas casser les données existantes.

---

36 Mock IA

37 Tests du Mock IA (Simulation Hors-Ligne)

Fichier source associé : `test/test_mock_ai.py`

37.1 Description

Ce fichier contient la suite de tests automatisés validant le module de simulation de l'intelligence artificielle (`mock_llm.py`), l'interprétation des variables de configuration d'environnement, et la bascule dynamique via la variable `MOCK_AI`.

37.2 Ce qui est testé

1. **`test_parse_bool`** : vérifie la robustesse du parsing des valeurs booléennes issues de l'environnement (`true`, `TRUE`, `1`, `yes`, `oui`, `false`, `0`, etc.).
2. **`test_mock_generate_commit_empty_diff_raises`** : valide qu'une exception `ValueError` est levée lorsqu'aucun diff n'est fourni.
3. **`test_mock_generate_commit_static_message`** : vérifie la conformité Conventional Commits du message statique généré (`feat(mock): commit mocker pour test hors ligne`).
4. **`test_mock_generate_commit_with_user_feedback`** : contrôle l'injection de la consigne utilisateur dans le corps du message lors d'une révision.
5. **`test_mock_call_chat`** : vérifie la structure de la réponse JSON simulée de l'API de chat Ollama.

6. **test_mock_release_notes** : confirme la génération d'une note de version Markdown statique déterministe.
7. **test_mock_ping** : contrôle la simulation d'un ping réussi vers Ollama avec latence factice et liste de modèles installés.
8. **test_llm_check_ollama_reachable_when_mock_ai** : valide que le ping rapide répond immédiatement sans requête HTTP lorsque `MOCK_AI=True`.
9. **test_llm_generate_commit_delegates_to_mock_when_mock_ai** : valide l'aiguillage automatique de `generate_commit_message` vers le mock.
10. **test_core_action_generate_commit_with_mock_ai** : valide le fonctionnement de l'action `generate-commit` du service core sous mode mock.
11. **test_core_action_release_notes_with_mock_ai** : valide l'action `release-notes` du service core sous mode mock.
12. **test_core_action_ping_ollama_with_mock_ai** : valide l'action `ping-ollama` du service core sous mode mock.

37.3 Stratégie de test

Les tests s'exécutent entièrement hors-ligne sans nécessiter de serveur Ollama ni d'accès réseau. Ils s'appuient sur `unittest.mock.patch` pour isoler `config.MOCK_AI` et valider les deux branches d'exécution.

37.4 Exécution des tests

```
pytest test/test_mock_ai.py
```

---

38 Golden diffs

39 Fixtures Golden Diff

Fichier source associé : test/fixtures/golden_diffs.py

39.1 Description

Ce module fournit des scénarios de diff déjà préparés, appelés `GoldenDiff`, pour simuler des changements de développement classiques et tester le bon classement des modifications par type de commit.

39.2 Structure de données

Chaque fixture contient :

- un `name` de scénario ;
- un `expected_type` correspondant au type attendu (`feat`, `fix`, `docs`, `refactor`, `test`, etc.) ;
- une liste de `DiffFile` représentant les fichiers modifiés.

39.3 Rôle fonctionnel

Ces fixtures servent de références pour valider la capacité du système à reconnaître la nature d'un changement à partir de son diff, sans dépendre d'un contexte réel de développement.

39.4 Exemple de scénarios

Parmi les cas couverts, on trouve :

- une fonctionnalité ajoutée (feature) ;
- une correction de bug ;
- une documentation mise à jour ;
- un refactoring ;
- l'ajout de tests ;
- une modification de dépendances ou de build.

39.5 Points de vigilance

- Les scénarios doivent rester représentatifs des modifications réelles du projet.
- Les types attendus servent de repères de validation, sans remplacer la logique métier du modèle.
- Les fixtures doivent rester stables pour assurer des tests reproductibles.

---

40 Initialisation

41 Activation des situations de test

Fichier source associé : test/situation_test/init_test.py

41.1 Description

Ce script réactive les éléments masqués pour les tests, en restaurant les fichiers et dossiers Git ou sensibles renommés avec le suffixe `_test`.

41.2 Rôle fonctionnel

Il sert à remettre dans leur état nominal les éléments suivants :

- `.git` → `.git`
- `.gitignore` → `.gitignore`
- `.github` → `.github`
- `.gitattributes` → `.gitattributes`
- `.gitmodules` → `.gitmodules`
- `.env` → `.env`

Le suffixe `_test` est utilisé pour neutraliser les éléments sensibles sans les supprimer du dépôt.

41.3 Comportement

Le script parcourt le dossier racine donné, recherche les entrées portant le suffixe `_test`, puis les renomme vers leur nom original. Il traite d'abord les éléments les plus profondément imbriqués pour éviter les conflits de renommage.

41.4 Utilité dans le projet

Cette étape est nécessaire lorsque les situations de test doivent être exécutées dans un environnement proche du vrai dépôt, notamment pour tester les interactions Git ou les fichiers de configuration sensibles.

---

42 Désinitialisation

43 Neutralisation des situations de test

Fichier source associé : test/situation_test/deinit_test.py

43.1 Description

Ce script masque les éléments Git et sensibles du projet en ajoutant le suffixe `_test` aux fichiers et dossiers concernés. Il s'agit du mécanisme inverse du script d'activation.

43.2 Rôle fonctionnel

Il protège l'environnement de travail en neutralisant automatiquement :

- `.git` → `.git_test`
- `.gitignore` → `.gitignore_test`
- `.github` → `.github_test`
- `.gitattributes` → `.gitattributes_test`
- `.gitmodules` → `.gitmodules_test`
- `.env` → `.env_test`

43.3 Objectif

Cette neutralisation empêche les outils externes, la plateforme de dépôt ou les scanners de sécurité de considérer les dossiers de test comme des ressources réelles du projet.

43.4 Comportement

Le script parcourt les sous-dossiers et renomme les éléments ciblés. Il traite les chemins les plus profonds en premier pour limiter les conflits de renommage lorsque plusieurs éléments sont imbriqués.

43.5 Utilité du projet

C'est une étape de sécurité et de reproducibilité essentielle pour les situations de test interactives, notamment dans des scénarios où le dépôt ou des variables d'environnement ne doivent pas être réellement exploités.

---

44 Déploiement de la documentation

45 Déploiement de la documentation

Cette page explique comment la documentation du projet est construite, publiée et mise à disposition via GitHub Pages.

45.1 Objectif

Le projet utilise MkDocs Material pour générer un site de documentation statique à partir des fichiers Markdown présents dans le dossier `docs/`. Une fois la documentation validée, elle est déployée automatiquement sur GitHub Pages.

45.2 Fichiers concernés

- `mkdocs.yml` : configuration du site MkDocs
- `.github/workflows/ci.yml` (jobs `documentation-build` et `deploy-docs`) : workflow GitHub Actions de build et déploiement
- `docs/` : contenu Markdown de la documentation

45.3 Configuration MkDocs

Le fichier `mkdocs.yml` définit :

- le nom du site : `SAE - Application intelligente`
- le thème : `material`
- la langue : `fr`
- la navigation du site via la clé nav

Le build MkDocs transforme les fichiers Markdown en un site statique dans le dossier `site/`.

45.4 Prérequis : activation manuelle de GitHub Pages (une seule fois)

!!! warning “À faire par un administrateur du dépôt” Avant le premier déploiement, GitHub Pages doit être activé manuellement, car le `GITHUB_TOKEN` du workflow n’a pas le droit de créer le site lui-même (erreur `Resource not accessible by integration` dans `actions/configure-pages` sinon) :

```
1. Aller sur **Settings > Pages** du dépôt.
2. Dans **Build and deployment**, mettre **Source = `GitHub Actions`**.
3. Relancer le workflow `CI` (automatique au prochain push sur `main`, ou manuel via **Actions
```

```
Le site est ensuite disponible à l'adresse configurée dans `mkdocs.yml` (`site_url`), soit <ht
```

45.5 Prérequis du PDF versionné

!!! warning “Important” Le hook `pre-commit` régénère `documentation.pdf` lorsqu’une page de documentation change. Pandoc et XeLaTeX doivent être installés, sinon le commit est bloqué.

```
```bash
sudo apt-get update && sudo apt-get install -y pandoc texlive-xetex texlive-
latex-recommended texlive-fonts-recommended lmodern
```
```

45.6 Workflow de déploiement

Le workflow GitHub Actions est défini dans `.github/workflows/ci.yml` (jobs `documentation-build` puis `deploy-docs`).

45.6.1 Déclenchement

Le build de documentation est exécuté lors de :

- un `push` sur toute branche (`branches: ['**']`)
- un `pull_request` ciblant `main` ou `dev`
- un déclenchement manuel via `workflow_dispatch`

Le déploiement officiel sur GitHub Pages (`deploy-docs` via `actions/deploy-pages@v4`) reste réservé à la branche `main`.

45.6.2 Marqueur de version de développement

Quand la documentation est générée depuis une branche autre que `main`, le titre du site reçoit un suffixe * pour signaler qu'il s'agit d'une version en cours de développement.

- * = documentation en cours de développement
- sans * = documentation officielle de la branche principale

45.6.3 Étapes du job `build`

1. `actions/checkout` : récupération du code source
2. `actions/setup-python` : installation de Python 3.12
3. installation de MkDocs Material
4. ajout du marqueur * pour les branches non principales
5. exécution de `mkdocs build`
6. publication d'un artefact de prévisualisation pour validation
7. publication sur GitHub Pages uniquement sur `main`

45.6.4 Étapes du job `deploy`

Le job `deploy` dépend du job `build` et ne se lance que pour `main`. Il publie le contenu sur l'environnement GitHub Pages via :

- `actions/deploy-pages@v4`

Cela permet d'exposer la documentation officielle en ligne, selon l'URL définie par le dépôt GitHub Pages.

45.7 Résultat attendu

Après un push ou un pull request sur `main` ou `dev`, GitHub Actions reconstruit la documentation. Les branches hors `main` affichent un suffixe * pour indiquer qu'il s'agit d'une version de développement. Seule la branche `main` est déployée publiquement sur GitHub Pages.

45.8 Bonnes pratiques

- garder les fichiers Markdown dans `docs/` cohérents avec la structure du projet
- mettre à jour `mkdocs.yml` si un nouveau chapitre ou une nouvelle page est ajouté
- valider le build localement avec `mkdocs build` avant de pousser

45.9 Commande de vérification locale

Pour construire le site en local :

```
mkdocs build
```

Cette commande crée le dossier `site/`, qui est le même contenu que celui déployé sur GitHub Pages.

---

46 Tests automatiques via GitHub Actions

47 Tests automatisés via GitHub Actions

Cette page décrit comment les tests du projet sont lancés automatiquement par les workflows GitHub Actions.

47.1 Objectif

Le projet utilise l'intégration continue pour vérifier automatiquement la qualité du code à chaque pull request et pour exécuter les tests importants dans un environnement standardisé.

47.2 Workflows disponibles

Un seul workflow est présent dans le dossier `.github/workflows/` :

- `ci.yml` : tests Go, tests Python et build/déploiement de la documentation (jobs `go-tests`, `python-tests`, `documentation-build`, `deploy-docs`)

47.3 Workflow CI

Le fichier `.github/workflows/ci.yml` est déclenché sur les `push` (toutes branches), les `pull_request` ciblant `main` et `dev`, et manuellement via `workflow_dispatch`.

47.3.1 Jobs

47.3.1.1 `python-tests`

Ce job exécute la suite de tests Python (module C) :

1. installation des dépendances depuis `requirements.txt` (dont `pytest`)
2. exécution de :

```
python -m pytest -q
```

`pytest.ini` exclut par défaut les tests marqués `slow` (ceux qui appellent le vrai serveur Ollama de l'IUT), via `addopts = -m "not slow"`. Le job CI ne lance donc que les tests mockés, qui ne dépendent d'aucune connexion réseau — c'est précisément pour ça que la distinction `slow` / non-`slow` existe.

47.3.1.2 `go-tests`

Ce job exécute les tests Go du projet :

1. configuration de Go à partir de `src/cli/go.mod`
2. exécution de :

```
go test ./...
```

Il est lancé depuis le dossier `src/cli` pour vérifier le sous-projet Go.

47.4 Vérification locale

Pour lancer les tests rapides localement :

```
python -m pytest -m "not slow" -q
```

Pour lancer les tests lents :

```
python -m pytest -m slow -q
```

Pour lancer les tests Go :

```
cd src/cli
go test ./...
```

47.5 Intérêt du workflow CI

Le workflow CI sert à :

- détecter les régressions rapidement
- vérifier les modifications avant fusion
- garantir une certaine qualité de code sur les branches principales
- conserver une exécution des tests lourds dans un cadre contrôlé

---

48 Sprint 00

49 Sprint N°1 : Mise en place du projet

User Stories :

- US 1.1 : En tant que développeur, je mets en place un pipeline d'intégration et de déploiement continus (CI/CD)
- US 1.2 : En tant que développeur, je choisis les différentes technologies à utiliser pour le projet (backend / frontend)
- US 1.3 : En tant que développeur, je crée des scénarios de test
- US 1.4 : En tant que développeur, je crée un client pour échanger avec le LLM

---

50 Sprint 01

51 Sprint N°2 : Intégration et développement des fonctionnalités

User Stories :

- US 2.1 : En tant que développeur, je corrige et adapte la récupération des données du `git diff` afin de les rendre compatibles avec le client LLM.

- US 2.2 : En tant que développeur, je mets en place un client Ollama permettant de générer un message de commit à partir des fichiers modifiés.
- US 2.3 : En tant que développeur, je mets en place la validation des réponses générées par le LLM afin de garantir le respect du format attendu.
- US 2.4 : En tant que développeur, je mets en place une gestion des erreurs et un système de nouvelle tentative lors de la génération du message de commit.
- US 2.5 : En tant que développeur, je définis les contrats `DiffFile` et `CommitMessage` permettant d'assurer la communication entre les différents modules.
- US 2.6 : En tant que développeur, je mets en place des tests unitaires et des tests d'intégration pour vérifier le fonctionnement du client LLM.
- US 2.7 : En tant que développeur, je mesure les performances et la conformité des réponses générées par le LLM.
- US 2.8 : En tant que développeur, je mets en place une architecture en miroir du projet afin de faciliter son organisation et sa documentation.
- US 2.9 : En tant que développeur, je réalise un wireframe permettant de définir la structure de l'interface utilisateur.
- US 2.10 : En tant que développeur, je réalise une première version de l'interface utilisateur en Go.

Livrables / DoR & DoD :
